

Solution Requirements

Date	29 September 2026
Team ID	6ab4ce760fc666a751b52130
Project Name	AI StudyBuddy
Maximum Marks	4 Marks

Define Functional and Non-Functional Requirements:

Create a comprehensive list of requirements to define exactly what your solution will do and how well it will perform.

- Functional Requirements (FR) define specific behaviors, functions, or features of the system (What the system should do).
- Non-Functional Requirements (NFR) define system quality attributes like performance, security, scalability (How well the system should do it).

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority
1	Authentication	Users register with name, email, and password. Passwords are hashed using bcrypt (10 salt rounds) before storage. Login returns JWT access token (15 min) and refresh token (7 days). Token refresh endpoint issues new token pairs without re-login.	High
2	Authorization levels	Two roles: "student" (default) and "admin". Students can only access/manage their own materials. Admins can view all users, delete users, view platform statistics, and access all materials. Role-Based Access Control enforced via middleware.	High
3	External interfaces	Integration with Google Gemini 2.5 Flash API via @google/generative-ai SDK for AI content generation (summaries, flashcards, quizzes, study plans). MongoDB Atlas cloud database for data persistence.	High
4	Transactions processing	File upload via multipart/form-data (Multer middleware). Supported formats: .txt, .md, .pdf (max 5MB). File content extracted as text, stored in MongoDB, temp file deleted. AI generation: material content sent as prompt to Gemini → response parsed and saved to material record.	High
5	Reporting	Admin dashboard endpoints: GET /api/admin/stats returns total user count and total materials count. GET /api/admin/users returns list of all registered users (passwords excluded).	Medium
6	Business rules, etc.	Students can only CRUD their own materials. Material ownership verified in controllers before any operation. AI features require a valid materialId. Flashcard/quiz count is configurable (default: 5). Study plan accepts goal, hoursPerDay, and days parameters.	High
7	Compliance to laws or regulations	Passwords never stored in plain text (bcrypt hashing). Sensitive keys (MONGO_URI, JWT secrets, GEMINI_API_KEY) stored in .env file excluded from version control via .gitignore. JWT tokens used for stateless authentication.	High
8	Other	Health check endpoint (GET /) returns server status. CORS enabled for frontend integration (default origin: http://localhost:3000). Global error handler middleware catches all unhandled errors. express-async-errors for automatic async error catching.	Medium

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	All API endpoints respond quickly. AI generation depends on Gemini API response time. Database queries optimized — list endpoints exclude heavy fields (content, flashcards, quiz, studyPlan).	API response < 2 seconds (excluding AI generation). AI generation < 10 seconds.
2	Scalability	MongoDB Atlas cloud database supports horizontal scaling. Modular MVC architecture allows independent scaling of components. RESTful API design enables future frontend integration (web, mobile).	Supports concurrent API requests. Cloud database scales on demand.
3	Security & Data Privacy	bcrypt password hashing (10 salt rounds). JWT dual-token authentication (access + refresh). Role-Based Access Control middleware. Material ownership checks in controllers. Environment variables for all secrets.	Zero plain-text password storage. All protected routes require valid JWT.
4	Reliability & Availability	express-async-errors catches all async errors automatically. Global error handler returns proper HTTP status codes and error messages. MongoDB Atlas provides built-in replication and failover.	Server handles errors gracefully without crashing. Database available 99.9% uptime (MongoDB Atlas SLA).
5	Usability & Accessibility	Complete API documentation with curl commands and Postman instructions. Clear JSON response format for all endpoints. Descriptive error messages for invalid requests.	All endpoints documented with request/response examples.
6	Other (Maintainability)	Modular folder structure: controllers, middleware, models, routes, utils. Nodemon for auto-restart during development. Clean separation of concerns following MVC pattern.	Code organized into logical modules. No circular dependencies.